Whitepaper about Plugin-Based Proactive Multimodal Interaction Trigger Architecture

jiating.fang

> ***Practice Foundation****: This architecture is based on the engineering practice of the open-source project* ***Prosophor*** *(C++ intelligent coding assistant CLI), and has been core implemented in that project.*
>
> ***Project URL****: https://github.com/Swair/prosophor*
>
> ***Theoretical Foundation****: The constraint analysis and design decisions of this architecture are based on theoretical results from "Theoretical Foundations of Proactive Interactive Agents", including the Triple Dilemma, Absence Detection Theorem, and Triple Convergence Argument.*

Abstract

LLM-driven agents have achieved remarkable progress in passive response, yet they remain fundamentally limited by user-initiated interaction paradigms—unable to proactively perceive environmental changes or anticipate user needs. This paper addresses the architectural bottlenecks that prevent proactive interaction: trigger logic tightly coupled with the kernel, absence of proactive triggering mechanisms, and lack of scalable plugin ecosystems. We propose a **Plugin-Based Proactive Multimodal Interaction Trigger Architecture** that decouples trigger logic from the scheduling kernel through a standardized three-file plugin format (`trigger`, `trigger_mode.cfg`, `ACTIVE.md`), enabling hot-swappable plugins with zero kernel modification. A dual-mode scheduling mechanism (periodic for continuous monitoring, idle for user-absence detection, with idle_once as a single-trigger variant) achieves graded, low-disturbance interaction while the Triple Dilemma constrains architectural trade-offs. The architecture has been implemented in the **Prosophor** project—a C++ intelligent coding assistant CLI—demonstrating engineering feasibility. Post-trigger LLM linkage generates scenario-aware responses across three modes: proactive dialogue, task execution with notification, and silent background execution. This paper contributes the first engineering framework for proactive agent interaction, bridging the gap between passive response systems and autonomous AI.

Keywords:proactive interaction; plugin architecture; trigger mechanism; Agent; multimodal interaction

2. Background and Pain Points

2.1 Development of Intelligent Agents

The development of intelligent Agents has gone through four key stages:

Table 1

Stage	Time	Characteristics
Early Exploration	1956—1995	Concept embodiment; Microsoft Bob, Clippy and other products attempted human-computer interaction practices
Differentiation	1996—2011	Product typification; software Agent concepts emerged but were limited by

		hardware and algorithms
Engineering Exploration	2012—2022	Deep learning breakthroughs; Task/Planning architecture emerged; Agent engineering began exploration
Standardization	2023—Present	LLM-driven Agent explosion; but proactive interaction architecture remains immature

2.2 Core Defects in Traditional Agent Design

Current mainstream Agent designs have the following **architectural-level problems** that constrain the implementation of proactive interaction capabilities:

Table 2

Defect	Specific Manifestation	Consequences
Passive Response Architecture	Agent Loop triggers after waiting for user commands	Unable to actively perceive scene changes and predict user needs
Trigger Logic Coupled with Kernel	Trigger conditions are hard-coded in the kernel	Adding/modifying trigger rules requires source code modification and recompilation
No Event Grading Mechanism	All events use the same trigger logic	Unable to balance emergency response and user do-not-disturb needs
Lack of Plugin Support	Does not support multi-script formats, high development threshold	Difficulty in functional extension, low plugin reusability
Lack of Ecosystem Loop	No unified review, distribution, and feedback channels	Plugins are developed in isolation, unable to form iterative optimization
No Proactive Trigger Capability	Agent Loop only supports passive calls	Unable to achieve "predict needs, actively respond" interaction mode

> ***Key Insight**: The root cause of the above problems does not lie in insufficient technical capabilities, but in **architectural design defects**—trigger logic coupling with the kernel, passive response architecture, and lack of ecosystem support.*

2.3 Design Goals of This Architecture

Addressing the architectural defects of traditional Agents, the design goals of this architecture are as follows:

Table 3

Goal	Traditional Agent Defect Addressed
Architecture Decoupling	Trigger logic coupled with kernel, extension requires source code modification
Proactive Triggering	Passive response architecture, unable to predict user needs
Graded Interaction	No event grading, unable to balance response and do-not-

	disturb
Flexible Invocation	Lack of plugin support, high development threshold
Ecosystem Loop	No unified ecosystem support, unable to iterate and optimize

3. Related Work

3.1 Rule Engine-Based Proactive Interaction Tools

Table 4

Product	Domain	Trigger Mode	Limitations
Hazel (macOS)	File Automation	Listen to file changes → Execute preset actions	Fixed rules, cannot extend, no intelligent response
AutoHotkey (Windows)	Desktop Automation	Hotkey/Event → Execute script	Script bound to configuration, no plugin ecosystem
IFTTT / Zapier	Web Service Automation	Webhook/Scheduled → Invoke API	Cloud rule engine, not local Agent architecture
Home Assistant	Smart Home	Sensor status → Linkage devices	Closed rule configuration, not plugin-based

> ***Key Insight**: These tools implement **event-based proactive triggering**, but use a **fixed rule engine** mode, cannot extend through plugins, and lack LLM linkage capabilities.*

3.2 Proactive Interaction Research Exploration

Table 5

Research Direction	Representative Work	Core Idea	Limitations
Proactive AI Assistant	Microsoft Cortana (2014-2020)	Time/location-based proactive reminders	Rule-driven, no architectural-level support
Context-Aware Agent	Academia (e.g., Context-Aware Computing)	Perceive environmental context, adaptive response	Research prototype, no engineering implementation
Ambient Intelligence	EU Research Projects	Ambient intelligence, imperceptible interaction	Concept ahead of time, insufficient engineering

3.3 Positioning of This Architecture

Table 6

Dimension	Traditional Agent	Rule Engine Tools	This Architecture
Trigger Mode	Passive response (user command)	Fixed rules (if-then)	**Plugin-based proactive trigger**
Extension Capability	Modify kernel/configuration	Modify script/rules	**Plugin hot-swapping**
Response Mode	LLM generation	Execute preset actions	**LLM scenario-based

			response**
Ecosystem Support	Skills/MCP (passive)	None	**Proactive interaction plugin community**

Relationship Between This Architecture and Prosophor Project:

• **Prosophor**: C++-based intelligent coding assistant CLI tool using REACT paradigm (Think-Act-Observe loop)

• **This Architecture**: Proactive interaction subsystem of Prosophor, embedded as a core module

• **Architecture Reusability**: Although derived from Prosophor practice, designed as a generic architecture that can be independently integrated into other Agent frameworks

> ***Key Insight**: This architecture is not pure theoretical research, but an **implementable solution derived from engineering practice and validated by projects**.*

4. Core Design Philosophy

With **"architecture decoupling as the core, proactive triggering as the breakthrough, graded low disturbance as the support, and ecosystem loop as the guarantee"**, reconstruct the Agent proactive interaction trigger system through architectural-level innovation.

Architecture Trade-off Note: According to the **Triple Dilemma** (see "Theoretical Foundations of Proactive Interactive Agents", Chapter 5), autonomy, low overhead, and real-time cannot be simultaneously satisfied. This architecture chooses:

• **Abandon autonomy** (perception goals are predefined) → Achieved through pluginization

• **Exchange for low overhead + real-time** (periodic polling + event-driven hybrid) → Achieved through dual-mode scheduling

Core Philosophy Comparison:

Table 7

Dimension	Traditional Agent Design	This Architecture Design
Architecture Mode	Monolithic coupling	Three-layer decoupling
Trigger Mode	Passive response (user command-driven)	Proactive prediction (event/state-driven)
Extension Mode	Modify source code + recompile	Plugin hot-swapping + zero kernel modification
Event Processing	Unified processing, no grading	Dual-mode grading (periodic/idle, idle_once as variant)
Ecosystem Support	None/scattered	Plugin community loop
Iteration Capability	Depends on version updates	Plugin autonomous iteration + feedback loop

5. Overall Architecture Design

This architecture is divided into two parts: **runtime architecture** and **ecosystem support system**:

• **Runtime Architecture** (two-layer innovation + one-layer reuse):

• **Plugin Layer** (Innovation): Trigger logic decoupled from the kernel, supporting hot-swapping

• **Scheduling Layer** (Innovation): Dual-mode graded triggering (periodic/idle, idle_once as variant), active perception and scheduling

• **Execution Layer** (Reuse): Reuses existing Agent framework tool execution capabilities (ReAct Act part)

• **Ecosystem Support System**: Plugin community (server-side + client-side)—independent infrastructure

5.1 Architecture Comparison: Traditional Agent vs This Architecture

```
┌──────────────────────────────────────────────────────────┐
│                Traditional Agent Architecture (Coupled)       │
│  ┌──────────────────────────────────────────────────┐   │
│  │  Kernel Layer: Trigger + Scheduling + Execution       │   │
│  │  Plugins: Fixed functionality, extension requires     │   │
│  │            kernel source modification               │   │
│  └──────────────────────────────────────────────────┘   │
│  → Problems: Difficult extension, high maintenance cost,    │
│              unable to proactively trigger                  │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│           Plugin Community (Ecosystem, Independent)          │
│           Plugin Upload, Security Check, Review,            │
│           Distribution, Update                              │
└──────────────────────────────────────────────────────────┘
                           │ Download/Update
                           ▼
┌──────────────────────────────────────────────────────────┐
│                 This Runtime Architecture (Three Layers)    │
│  ┌──────────────────────────────────────────────────┐   │
│  │  Plugin Layer (Innovation): Trigger logic decoupled  │   │
│  │     active/ directory, trigger + trigger_mode.cfg    │   │
│  │     → All trigger logic sinks to plugins, zero       │   │
│  │       coupling with scheduling kernel                │   │
│  └──────────────────────────────────────────────────┘   │
│                            │                               │
│                            ▼                               │
│  ┌──────────────────────────────────────────────────┐   │
│  │  Scheduling Layer (Innovation): Dual-mode graded     │   │
│  │     triggering (periodic/idle), active perception & scheduling       │   │
│  │     Plugin scan, classified invocation, priority     │   │
```

```
│  │     scheduling, LLM linkage                          │   │
│  │     → Only responsible for scheduling, does not      │   │
│  │       contain specific trigger logic                 │   │
│  └──────────────────────────────────────────────────────┘   │
│                           │                                 │
│                           ▼                                 │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  Execution Layer (Reuse): Reuses Agent framework     │   │
│  │     tool execution capabilities                      │   │
│  │     Scenario interaction, multimodal output,         │   │
│  │     configurable (ReAct Act)                         │   │
│  └──────────────────────────────────────────────────────┘   │
│  → Innovation Focus: Plugin Layer + Scheduling Layer,       │
│    Execution Layer reuses existing framework                │
└─────────────────────────────────────────────────────────────┘
```

5.2 Ecosystem Support System: Plugin Community

The plugin community is an infrastructure independent of the runtime architecture, providing ecosystem support for the architecture:

Table 8

Component	Function
Server-side	Plugin upload, security check, manual review, classified storage, version management, statistics
Client-side (embedded in runtime)	Plugin search, one-click download, automatic deployment, version update, feedback submission
Value	Build a complete ecosystem loop of "development→review→distribution→use→feedback→iteration"

Differentiated Positioning:

Table 9

Ecosystem Type	Representative Products	Service Objects	Plugin Type
General Plugin Ecosystem	OpenAI Skills, Claude MCP	Passive Response Agents	Passive tool-type plugins
Proactive Interaction Plugin Ecosystem	This Plugin Community	**Proactive Interaction Agents**	**Trigger rule-type plugins**

> ***Note**: There are already general plugin ecosystems (such as Skills, MCP), but they are **passive tool-type** plugins that require waiting for user or Agent invocation. This plugin community focuses on **proactive trigger rule-type** plugins, filling the gap in the proactive interaction field.*

5.3 Runtime Architecture Layers Description

5.3.1 Plugin Layer (Business Logic Layer)

Traditional Agent Problem: Trigger logic is hard-coded in the kernel, extension requires source code modification.

This Innovation:

• **Storage Directory**: Global unified root directory `~/.prosophor/active/`, all plugins are independent subdirectories under this directory

• **Plugin Composition**: Each plugin must contain 3 core files:

  • `trigger` (trigger script, any language, returns true/false)

  • `trigger_mode.cfg` (mode configuration file: periodic/idle, with idle_once as idle variant)

  • `ACTIVE.md` (interaction configuration document for LLM linkage)

Plugin Example (CPU Temperature Monitoring Plugin):

```
~/.prosophor/active/
└── cpu_temperature_monitor/
    ├── trigger          # Trigger script (any language, returns true/false)
    ├── trigger_mode.cfg # Mode configuration
    └── ACTIVE.md        # Interaction configuration
```

trigger Trigger Logic Flow:

```
┌──────────────────────────────────────────────────────────┐
│  trigger Execution Flow                                  │
├──────────────────────────────────────────────────────────┤
│  1. Execute detection logic (e.g., read CPU temperature)  │
│          │                                               │
│          ▼                                               │
│  2. Judge whether trigger condition is met               │
│          │                                               │
│      ┌───┴────┐                                          │
│      │ Yes    │ No                                       │
│      ▼        ▼                                          │
│  Return true  Return false                               │
│  (triggered)  (not triggered)                            │
│          │                                               │
│          ▼                                               │
│  3. Output trigger reason to stdout (optional)           │
│     Example: "CPU temperature 90°C, exceeds threshold 85°C" │
└──────────────────────────────────────────────────────────┘
```

Output Specification:

• Return `true` = triggered (can print trigger reason to stdout)

• Return `false` = not triggered

• Timeout `100ms` forced termination to avoid blocking scheduling

trigger_mode.cfg Configuration:

Table 10

Field	Description	Example Value
mode	Trigger mode	periodic / idle (with idle_once variant)
interval	Execution interval (seconds)	60
priority	Priority (0-100, smaller = higher)	10

ACTIVE.md Configuration:

Table 11

Field	Description
Trigger Scenario	Describe trigger conditions and event type
Interaction Script	Reference template for LLM to generate callback
Executable Operations	List of tasks users can execute with one click

Post-Trigger Flow:

```
trigger returns true → Read ACTIVE.md → Send to LLM → Generate response → Notify user/Execute task
```

• **Plugin Characteristics**: Supports hot-swapping, adding/modifying/uninstalling plugins does not require restarting the architecture or modifying kernel source code

5.3.2 Scheduling Layer (Core Hub Layer)

Traditional Agent Problem: Scheduling logic is coupled with trigger logic, unable to flexibly adapt to different trigger scenarios.

This Innovation:

• **Plugin Scan & Load**: Traverse plugin directory, verify plugin integrity, classify and store in corresponding plugin pools; supports hot-loading, adding/removing plugins without restart

• **Dual-Mode Graded Scheduling**: Differentially invoke by trigger type (periodic/idle, with idle_once as idle variant)

• **Script Control & Result Parsing**: Subprocess isolated invocation, strictly parse trigger state (true=trigger, false=no trigger)

• **Priority Scheduling**: Mandatory plugins take priority over idle plugins to resolve concurrency conflicts

• **LLM Linkage**: After successful trigger, read ACTIVE.md and send to LLM, receive callback response and dispatch to execution layer

5.3.3 Execution Layer (Interaction Implementation Layer)

Note: The execution layer reuses existing Agent framework tool execution capabilities (such as ReAct's Act part), this paper does not innovate.

This Focus:

• **Plugin Layer**: Trigger logic decoupled from kernel, supports plugin-based extension

• **Scheduling Layer**: Dual-mode graded triggering (periodic/idle, idle_once as variant), achieves proactive interaction

• **Ecosystem Support**: Plugin community loop, supports large-scale engineering implementation

6. Core Function Implementation

6.1 Proactive Interaction Implementation: Dual-Mode Graded Triggering

Proactive Interaction Core Philosophy: Upgrade from "passive response" to "proactive prediction."
Difference from "Continuous Sampling Input Proactive Interaction":

Table 12

Dimension	Continuous Sampling Input (e.g., File Monitor, Hazel)	This Architecture
Positioning	**Perception Technical Means**: Polling/listening system state	**Engineering Framework**: Accommodates various perception methods
Trigger Logic	Fixed rules: if change → then action	Plugin-based: trigger returns true/false
Response Mode	Execute preset actions	Read ACTIVE.md → LLM generates response → Notify/Execute
Extension Mode	Requires modifying code/configuration rules	Add plugins, no kernel modification
Ecosystem Support	None, used in isolation	Plugin community loop
Iteration Capability	None, rules solidified	Plugins can be updated, feedback iteration

> ***Note**: Continuous sampling input (such as listening to files, mouse, system state) is a **perception technical means** and can be used as an implementation method for plugins in this paper. However, the core value of this architecture lies in providing an **engineering framework for proactive interaction**—the perception method is determined by the plugin, and after triggering, it links with the LLM to generate intelligent responses and achieves large-scale extension through the plugin ecosystem.*

Interaction Subjectivity Analysis: User presets trigger rules, system proactively initiates interaction

Table 13

Dimension	Traditional Passive Interaction	This Architecture Interaction
Trigger Subject	User command	Plugin script (extension of user will)
Interaction Initiator	User	System (Agent)
Example	User inputs `/help` → Agent responds	trigger returns true → Agent proactively notifies user

Interaction Flow:

```
User Preset Phase:
  User downloads/writes plugin → Registers to `~/.prosophor/active/` directory → Configures trigger conditions
  ↓
  Runtime Phase:
```

```
  Scheduler scans plugins → trigger script returns true → LLM generates response → Proactively notifies user
```

> ***Note**: This is **user-preset system proactive interaction**, not LLM "autonomous consciousness" style proactive interaction. Plugin scripts are an extension of user will, and the LLM's "proactivity" is reflected in: when user-preset conditions are met, it generates responses **without requiring further user commands**. Analogy: Phone alarm clock—user sets the alarm, and the alarm proactively reminds the user when it rings.*

6.1.1 Proactive Interaction Hierarchy Positioning

The "proactive interaction" of this architecture belongs to **engineering-feasible conditional proactivity**, distinct from sci-fi style "AI autonomous consciousness" proactive interaction.

Proactive Interaction Hierarchy:

```
┌──────────────────────────────────────────────────────────┐
│  Proactivity Levels                                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ① Fully Passive ← Traditional Agents (ChatGPT):         │
│      Waiting for user commands                           │
│                                                          │
│  ② Conditional Proactive ← This Architecture:            │
│      User presets conditions, system proactively initiates │
│                                                          │
│  ③ Autonomous Proactive ← Sci-Fi AI:                     │
│      Agent autonomously perceives, decides, and serves   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Hierarchy Comparison:

Table 14

Dimension	① Fully Passive	② Conditional Proactive (This Paper)	③ Autonomous Proactive
Interaction Timing	User command	Condition met	Agent autonomous judgment
Trigger Decision Subject	User	User preset + System execution	Agent autonomous
Response Content Generation	Agent generated	Agent generated	Agent generated
Proactivity Level	Fully passive	Conditional proactive	Fully proactive
Engineering Feasibility	✅Mature	✅Feasible	□ Technology not mature

This Architecture Positioning:

- ✅**Interaction Timing Proactive**: No need for user to initiate command
- ✅**Response Generation Proactive**: LLM generates scenario-based response

- □ **Trigger Condition Preset**: User defines concerns, not Agent autonomous judgment

> ***Boundary Note**: This architecture is **engineering-feasible Agent proactive interaction**, achieving a balance between "user preset conditions" and "system proactive execution", avoiding over-interpretation as "AI autonomous consciousness".*

Dual-Mode Graded Triggering Mechanism (Comparison with Traditional Agents):

Table 15

Dimension	Traditional Agent	This Architecture
Trigger Mode	Single: Only supports user command triggering	Dual-mode: periodic/idle (idle_once as variant)
Event Grading	No grading, unified processing	Graded: Critical (mandatory) + normal (idle)
Do-Not-Disturb Support	Not supported	Supported: Interrupt normal triggering when user interacts
Single Trigger Support	No	Supported: idle_once (one-time guidance)
Timeout Protection	None	Supported: Script timeout 100ms to avoid blocking

Trigger Modes:

Table 16

Mode	Trigger Condition	Applicable Scenarios	Safeguard Mechanism
Periodic Mandatory Trigger (periodic)	Trigger periodically according to configured interval	Critical events (hardware failures, etc.)	Script timeout 100ms to avoid blocking scheduling
User Idle Trigger (idle)	Trigger when no interaction with LLM reaches threshold	Normal events (idle reminders, etc.)	Immediately interrupt when user resumes interaction
Single Idle Trigger (idle_once)	Trigger once when no interaction with LLM reaches threshold	One-time suggestions (e.g., first-time user guidance)	Triggers once per idle period, resets when user re-interacts

Configuration Example (`trigger_mode.cfg`):

```
# ========================================
# Periodic Mandatory Trigger (Critical Event)
# ========================================
mode=periodic
interval=60        # Trigger once every 60 seconds
priority=10        # Priority: 0-100, smaller number = higher priority
timeout=100        # Script timeout 100ms

# ========================================
```

```
# User Idle Trigger (Normal Event)
# ========================================
mode=idle
threshold=300      # Trigger after 300 seconds of no interaction with LLM
interval=60        # Execute once every 60 seconds while idle
priority=50        # Medium priority
timeout=100

# ========================================
# Single Idle Trigger (One-time Guidance)
# ========================================
mode=idle_once
threshold=120      # Trigger after 120 seconds of no interaction with LLM
priority=40        # Medium-low priority
timeout=100
# Note: Triggers once per idle period, resets when user re-interacts, triggers again on next
idle
```

Priority Explanation:

Table 17

Priority	Range	Description	Example Scenarios
Critical	0-10	System-level alarm, immediate execution, can interrupt user operations	Hardware failure, security threat
High	11-30	Important notifications, priority scheduling	Critical task reminders
Medium-High	31-50	Normal reminders, execute during user idle time	File organization, cache cleanup
Medium	51-70	Suggestion notifications, low disturbance	Usage tips recommendations
Low	71-100	Background tasks, lowest priority	Data synchronization, log archiving

6.1.2 Multi-Period Independent Scheduler

The scheduling layer adopts a **multi-period independent scheduling strategy**, triggering each periodic plugin independently according to its configured `interval`, achieving precise scheduling and low CPU idle usage.

Core Design:

```
┌──────────────────────────────────────────────────────────────┐
│              Multi-Period Independent Scheduling Diagram      │
├──────────────────────────────────────────────────────────────┤
│                                                               │
```

```
│  Plugin Config           Scheduling Behavior                  │
│  ┌──────────────┐                                              │
│  │ Plugin A     │──► Trigger every 10 seconds                  │
│  │ interval=10  │                                              │
│  └──────────────┘                                              │
│                                                                │
│  ┌──────────────┐                                              │
│  │ Plugin B     │──► Trigger every 60 seconds                  │
│  │ interval=60  │                                              │
│  └──────────────┘                                              │
│                                                                │
│  ┌──────────────┐                                              │
│  │ Plugin C     │──► Trigger every 300 seconds                 │
│  │ interval=300│                                              │
│  └──────────────┘                                              │
│                                                                │
│  Scheduler: Calculate earliest due time → Sleep → Wake &     │
│             Execute → Loop                                     │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

Scheduling Strategy Comparison:

Table 18

Strategy	Implementation	Advantages	Disadvantages
Unified Scan Cycle	Fixed cycle (e.g., every 10 seconds) scans all plugins, checks if due	Simple implementation	CPU waste from idle spinning, low scheduling precision
Independent Period Trigger	Each plugin triggers independently according to its own `interval`	Precise scheduling, low CPU usage	Requires timer management

This architecture adopts independent period trigger strategy, with advantages:

```
┌────────────────────────────────────────────────────────────────┐
│  Independent Period Scheduling Advantages                      │
├────────────────────────────────────────────────────────────────┤
│                                                                │
│  ✅Precise Triggering: Each plugin executes independently     │
│     according to its own period, no interference               │
│                                                                │
│  ✅Low CPU Usage: Timer sleep-wake mechanism, does not        │
│     occupy CPU                                                 │
│                                                                │
│  ✅High Extensibility: New plugins dynamically added to       │
```

```
│     scheduling queue, no restart required                    │
│                                                              │
│  ✅Priority Assurance: Critical plugins can be scheduled    │
│     with higher priority                                     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

6.1.3 Idle Trigger

Idle trigger ensures "no disturbance during user focus, proactive service during idle moments."
Idle Definition: "User idle" in this architecture refers to **no interaction with the LLM**, including:

• No conversation request (user has not sent new messages)

• No task execution (user has not triggered Agent tasks)

• No feedback operations (user has not rated/operated on LLM responses)

> ***Theoretical Support**: According to the **Absence Detection Theorem** (see "Theoretical Foundations of Proactive Interactive Agents", Chapter 4), "user idle" is an absence concept (no input events), which cannot be detected by pure event-driven approach and must rely on time reference (threshold) for judgment.*
>
> ***Note**: Using "interaction with LLM" rather than "system input events" as the basis for idle judgment avoids collecting privacy data such as mouse/keyboard input, and more accurately reflects the user's attention state toward the Agent.*

Working Flow:

```
Detect idle → Countdown threshold → Trigger → Execute at interval → User re-interacts → Reset
→ Next idle cycle
```

Parameters:

Table 19

Parameter	Controls	Adjustment Effect
`threshold`	Idle mode entry threshold	Larger = less sensitive, smaller = more sensitive
`interval`	Execution frequency in idle mode	Larger = less frequent, smaller = more frequent

Configuration Example (`trigger_mode.cfg`):

```
# User Idle Trigger (Enhanced Configuration)
mode=idle
threshold=300      # No interaction with LLM for 300 seconds, enter idle trigger mode
interval=60        # After entering idle mode, execute trigger script every 60 seconds
priority=50        # Priority
timeout=100        # Script timeout 100ms
```

Interaction Event Types:

Table 20

Event Type	Description	Example
Conversation Request	User actively sends message	User inputs "/help"
Task Execution	User triggers Agent task	User clicks "Execute" button
Feedback Operation	User operates on response	Like/Dislike/Copy response

Dual-Parameter Independent Control:

```
┌──────────────────────────────────────────────────────────────┐
│  Configuration Combination Examples                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Scenario 1: Light Disturbance (e.g., suggestion notifications)│
│  threshold=600, interval=300                                 │
│  → Trigger after 10 minutes of no interaction, detect every  │
│    5 minutes after triggering                                │
│                                                              │
│  Scenario 2: Medium Disturbance (e.g., file organization     │
│  reminder)                                                   │
│  threshold=300, interval=120                                 │
│  → Trigger after 5 minutes of no interaction, detect every   │
│    2 minutes after triggering                                │
│                                                              │
│  Scenario 3: Heavy Disturbance (e.g., sedentary reminder)    │
│  threshold=120, interval=60                                  │
│  → Trigger after 2 minutes of no interaction, detect every   │
│    minute after triggering                                   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

idle_once is idle with a `triggered_count=1` parameter — triggers once per idle period, resets when user re-interacts, triggers again on next idle.

6.1.4 Trigger Modes Comparison

Table 21

Mode	Trigger Prerequisite	Trigger Count	Applicable Scenarios	Configuration Complexity
periodic	None, triggered by interval	Infinite	Continuous monitoring (CPU, memory)	Low
idle	No interaction with LLM ≥ threshold	Multiple during idle	Idle suggestions (cache cleanup)	Medium

idle_once	No interaction with LLM ≥ threshold	Once per idle period	One-time guidance (new userhint)	Low

Selection Guidelines:

Quick Configuration:

Table 22

Configuration Method	Description	Example
`threshold=0`	Skip idle detection, directly trigger periodically	`mode=idle, threshold=0` → Equivalent to periodic
`threshold>0`	Enable idle detection	`mode=idle, threshold=300`
`mode=idle_once`	Triggers once per idle, resets on re-interaction	`threshold=120`

> ***Note**: When `threshold=0` in idle mode, idle detection is automatically skipped, equivalent to periodic mode, no need to manually select trigger mode.*

6.2 Plugin Implementation (Comparison with Traditional Solutions)

Traditional Solutions Have No Proactive Plugin Design: Traditional tools (Hazel, AutoHotkey, file monitors, etc.) use a **fixed rule engine** mode, rules are bound to programs, and proactive triggering capabilities cannot be extended through plugins.

Table 23

Dimension	Traditional Solutions (Hazel, AutoHotkey, etc.)	This Architecture
Architecture Mode	Fixed rule engine (if-then configuration)	Proactive plugin framework
Trigger	No standardized format, rules hard-coded in configuration	Standardized trigger script (returns true/false)
Extension Mode	Modify configuration or write new scripts	Add plugins, no kernel modification
Response Mode	Execute preset actions	LLM generates scenario-based response
Loading Mode	Takes effect after restart	Hot-loading (30-second automatic detection)
Ecosystem Support	None, used in isolation	Plugin community loop

Core Value of This Paper: The first plugin-based framework for **proactive interaction scenarios**, upgrading scattered proactive trigger rules into a shareable and iterable engineering system.

6.3 LLM Linkage Implementation

Core Flow After Trigger:

```
trigger returns true → Read ACTIVE.md → Create temporary session → Agent Loop → Execute mode
```

Three Response Modes:

Types A/B/C are typical examples of proactive interaction modes; the architecture supports infinite extension through plugin design.

```
┌──────────────────────────────────────────────────────────────────┐
│  After trigger returns true: Execute one of three modes          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  1. Create temporary session                                     │
│  2. Append system prompt from ACTIVE.md                          │
│  3. Append user message: "Trigger: <trigger_reason>"             │
│  4. Invoke Agent Loop                                            │
│  5. After LLM response, execute one of three modes:              │
│                                                                  │
│     ┌──────────────────┬──────────────────┬──────────────────┐
│     ▼                  ▼                  ▼                  │
│  Type A: Proactive   Type B: Execute     Type C: Silent        │
│  Interaction           Task + Notify       Execution           │
│  - LLM generates       - Agent executes    - Agent executes    │
│    reply content         tasks via           tasks silently    │
│  - Interact with         ToolRegistry      - No notification   │
│    user (dialogue +    - Notify user       - Session destroyed │
│    emotional exchange)   upon completion   - Optional logging  │
│  - Session destroyed   - Session destroyed                     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

> ***Note**: Three response types are **orthogonal** to two trigger modes (periodic/idle)—any trigger mode can be paired with any response type.*

Table 24

Mode	Description	Example	Applicable Scenarios
Type A: Proactive Interaction	LLM generates reply content, interact with user (supports emotional exchange)	"CPU temperature too high, recommend checking cooling fan" / "Remember to take a break when working hard"	Warnings, suggestions, reminders, emotional support
Type B: Execute Task + Notify	Agent executes tasks via ToolRegistry based on ACTIVE.md configuration, notifies user upon completion	"Cleaned 5 background processes, CPU temperature dropped 3°C"	Tasks requiring user awareness (cleanup, maintenance)
Type C: Silent Execution	Agent executes tasks silently based on ACTIVE.md configuration, no notification	Background cleanup, scheduled sync	Background routine tasks (no user disturbance needed)

6.3.1 Type A: Proactive Interaction (Dialogue + Emotional)

• **Essence**: LLM as "dialoguer", generates reply content to interact with user, supports emotional exchange

• **Flow**: Trigger → Create temporary session → LLM generates response → UI display → Destroy

• **Characteristics**: Lightweight, non-blocking, supports emotional exchange (care, reminders, encouragement, etc.)

• **Scenarios**: Warnings, suggestions, reminders, emotional companionship, one-time guidance

6.3.2 Type B: Execute Task + Notify (Task Orchestration)

• **Essence**: Agent as "executor", orchestrates and executes tasks based on ACTIVE.md configuration

• **Flow**: Trigger → Create temporary session → Agent executes via ToolRegistry → Summary notification → Destroy

• **Characteristics**: Complete tool call chain, task orchestration, progressive notification, quantifiable results

• **Scenarios**: Tasks requiring user awareness (system cleanup, file archiving, data backup, etc.)

6.3.3 Type C: Silent Execution (Background Routine)

• **Essence**: Agent as "background service", completes tasks without user perception

• **Flow**: Trigger → Create temporary session → Agent executes → No notification → Optional logging → Destroy

• **Characteristics**: Zero disturbance, background execution, optional logging for post-audit

• **Scenarios**: Scheduled cleanup, data synchronization, log rotation, health checks, and other routine tasks

6.3.4 Three Response Modes Comparison

Table 25

Dimension	Type A (Dialogue)	Type B (Task)	Type C (Silent)
Agent Role	Dialoguer/Assistant	Executor/Steward	Background Service
Tool Calling	No	Yes	Yes
User Notification	Yes (interaction/emotional)	Yes (result summary)	No
User Perception	Strong perception	Medium perception	No perception
Typical Scenarios	Warnings/Suggestions/Companionship	Tasks requiring awareness	Background routine tasks

Mode Selection Principles:

• **Type A**: User needs to be informed or needs emotional value

• **Type B**: Task needs execution and user needs to know the result

• **Type C**: Routine tasks, no need to disturb user

6.4 Plugin Ecosystem Loop Implementation

Traditional Agent Problem: No ecosystem support, plugins are developed in isolation, unable to form iterative optimization.

This Core Support:

• **Server-side**: Plugin upload, security review, classification management, version iteration, feedback collection

• **Client-side**: Plugin search, one-click download, automatic update, local management, feedback submission

• **Offline Adaptation**: Supports offline use of local plugins, automatically syncs data after network recovery

7. Proactive Interaction Value and Application Scenarios

7.1 Core Value: Three Major Breakthroughs from Traditional Agents to This Architecture

Table 26

Breakthrough	From... To...	Description
1	**Passive Response → Proactive Prediction**	No need for users to actively initiate commands, the system autonomously perceives scene changes and predicts needs
2	**Scattered Triggering → Loop Iteration**	Combines Agent Loop to record execution results and user feedback, completing iterative optimization
3	**Fixed Functionality → Plugin Extension**	Plugin framework + ecosystem loop, supports large-scale business scenario extension

7.2 Typical Application Scenarios

Table 27

Scenario	Application Mode	Traditional Agent Limitations	This Architecture Advantages
Industrial Control	Periodically detect CPU temperature, memory usage, proactively trigger fault warnings	Requires user to actively query or configure complex rules	Plugin configuration, proactive warning
Desktop	Proactively remind file organization, cache cleanup during user idle time	Unable to judge user state, easily disturbing	Idle triggering, interrupt immediately when user resumes interaction
IoT	Automatically detect temperature, humidity, light, proactively link devices for control	Depends on fixed rules, unable to adapt to scenarios	LLM linkage, scenario-based response
Intelligent Agent	Supplement proactive triggering capability for	Agent Loop only supports passive calls	Proactive triggering + loop iteration

	Agent Loop		

7.3 Engineering Practice and Outlook

The architecture has been core implemented in the Prosophor project. Future work will focus on standardization, scenario library construction, and active plugin ecosystem development.

8. Conclusion and Outlook

8.1 Innovation Summary

The core breakthrough of this architecture lies in **architectural-level innovation**, proposing systematic solutions for the core defects of traditional Agent design:

Table 28

Traditional Agent Defect	This Innovation Solution	Implementation Effect
Trigger logic coupled with kernel	Plugin framework, trigger logic sinks to plugin layer	Plugin hot-swapping, zero kernel modification extension
Passive response architecture	**Proactive interaction framework** (dual-mode graded triggering)	Proactive prediction, balances emergency response and do-not-disturb
No ecosystem loop support	Plugin community (development→review→distribution→feedback)	Sustainable iterative engineering system
Extension requires source code modification	Standardized plugin specification (3 core files)	Low-threshold development, large-scale extension

Core Innovations:

Proactive Interaction (Core Value): The industry's first proactive interaction trigger framework—upgrading from "passive response" to "proactive prediction". Supports three typical interaction modes (extensible via plugins):

- **Type A: Proactive Interaction**: LLM generates reply content to interact with user, supports emotional exchange (e.g., "CPU temperature too high, recommend checking cooling fan" / "Remember to take a break when working hard")
- **Type B: Execute Task + Notify**: Agent executes tasks based on plugin configuration's ACTIVE.md via ToolRegistry, notifies user upon completion (e.g., "Cleaned 5 background processes, CPU temperature dropped 3°C")
- **Type C: Silent Execution**: Agent executes tasks silently based on plugin configuration, no notification to user (e.g., background cleanup, scheduled sync)

Plugin Framework (Architecture Support): Trigger logic decoupled from kernel + plugin hot-swapping—solving the industry-wide pain point of "trigger logic coupled with kernel"

Engineering Design (Large-scale Guarantee): Plugin community loop + standardized plugin specification—supporting large-scale extension and sustainable iteration

Dual-Mode Triggering (Technical Implementation): periodic/idle two trigger modes (idle_once as idle variant), covering all scenarios including continuous monitoring, idle suggestions, and one-time guidance

8.2 Outlook

Future optimization will focus on architectural innovation, advancing in three directions:

Standardization Promotion: Promote proactive interaction plugin specifications to become industry standards

Scenario Library Construction: Accumulate proactive trigger scenario templates, lower plugin development threshold

Active Plugin Ecosystem Development: Build proactive interaction plugin community, supporting "development→review→distribution→feedback→iteration" loop

References

Anderson, P. W. (1972). More Is Different. Science.

Brooks, F. P. (1995). The Mythical Man-Month.

Shannon, C. E. (1948). A Mathematical Theory of Communication.

Citation & License

DOI: 10.5281/zenodo.19762803
Licensed under [Creative Commons Attribution 4.0 International (CC BY 4.0)](https://creativecommons.org/licenses/by/4.0/)